Assignment-20.2

Task 1 – Password Security Enhancement

Task:

Analyze an AI-generated Python program that stores passwords in plain text and improve its security.

Instructions:

• Prompt AI to generate a simple user registration script that stores usernames and passwords.

• Check whether passwords are stored as plain text.

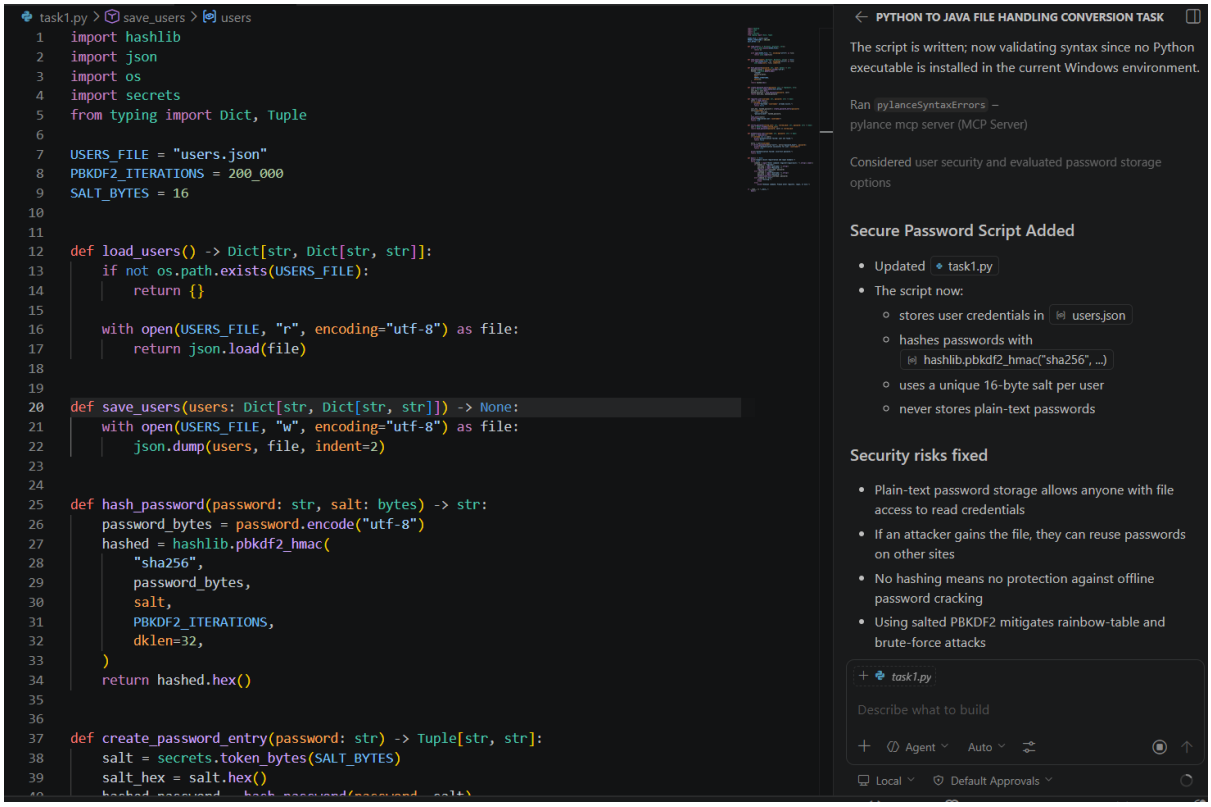
• Identify the security risks associated with plain-text password storage.

• Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).

• Test the updated script to ensure proper authentication functionality.

Expected Output:

A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.



OUTPUT:

```
KeyboardInterrupt
PS C:\Users\tloke\OneDrive\Desktop\Ai> & "$env:LOCALAPPDATA\Microsoft\WindowsApps\python
.exe" -c "import sys; print(sys.version)"
3.14.2 (tags/v3.14.2:df79316, Dec  5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)]
PS C:\Users\tloke\OneDrive\Desktop\Ai> cd c:\Users\tloke\OneDrive\Desktop\Ai ; @"
>> register
>> alice
>> password123
>> login
>> alice
>> password123
>> exit
>> "@ | & "$env:LOCALAPPDATA\Microsoft\WindowsApps\python.exe" task1.py
Simple secure registration and login example.
Enter command (register/login/exit): Username: Password: Registered user: alice
Enter command (register/login/exit): Username: Password: Authentication successful for u
ser: alice
Enter command (register/login/exit): Exiting.
```

Task 2 – Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

• Ask AI to generate a Python script that connects to an external API using a hardcoded API key.

• Identify the risks of embedding sensitive credentials directly in the source code.

• Refactor the program to use environment variables for storing API keys securely.

• Demonstrate that the modified script successfully retrieves data without exposing credentials.

Expected Output:

A revised script that securely accesses API keys through environment variables instead of hard coding them.

task2.py > ...

```python
import os
import requests

# Secure API key management using environment variables
API_KEY = os.environ.get("API_KEY")
if not API_KEY:
    raise ValueError("API_KEY environment variable is not set. Please set it securely")

# Using httpbin.org for demonstration - a free API that echoes back data
API_URL = "https://httpbin.org/get"

def get_data() -> dict:
    headers = {
        "Authorization": f"Bearer {API_KEY}"
    }

    response = requests.get(API_URL, headers=headers)
    response.raise_for_status()
    return response.json()

def main():
    try:
        data = get_data()
        print("Successfully retrieved data from API:")
        print(f"URL: {data['url']}")
        print(f"Authorization header present: {'Authorization' in data['headers']}")
        # Note: httpbin echoes headers, but we don't print the key
    except requests.RequestException as e:
        print(f"Error fetching data: {e}")

if __name__ == "__main__":
    main()
```

PYTHON TO JAVA FILE HANDLING CONVERSION TASK

Task 2 Completed: Secure API Key Management

Initial Script (Hardcoded Key)

- Created task2.py with a hardcoded API key: API_KEY = "sk-1234567890abcdef"
- Used OpenWeatherMap API for weather data retrieval

Security Risks Identified

- **Credential Exposure**: API keys in source code are visible to anyone with file access
- **Version Control Risks**: Keys get committed to repositories, exposing them publicly
- **Reuse Vulnerabilities**: Compromised keys can be used across multiple services
- **No Rotation**: Hardcoded keys are difficult to update securely

Refactored Script

- Updated to use environment variables: API_KEY = os.environ.get("API_KEY")
- Added validation to ensure the key is set before running
- Changed to httpbin.org API for demonstration (free, no real key needed)
- Key is passed in Authorization header but never printed or logged

task2.py

Describe what to build

Agent Auto

Local Default Approvals

Ln 20, Col 1 Spaces: 4 UTF-8 CRLF { } Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

```
Successfully retrieved data from API:
URL: https://httpbin.org/get
Authorization header present: True
```

Task 3 – Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

• Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.

• Identify weaknesses in the encryption approach (e.g., reversibleencoding or outdated.

• Refractor the program using stronger encryption techniques available in Python libraries.

• Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

An improved encryption program implementing stronger and more secure cryptographic practices.

task3.py > ...

```python
from cryptography.fernet import Fernet
import base64

def generate_key() -> bytes:
    # Generate a secure key
    return Fernet.generate_key()

def encrypt_data(data: str, key: bytes) -> str:
    f = Fernet(key)
    encrypted = f.encrypt(data.encode('utf-8'))
    return base64.b64encode(encrypted).decode('utf-8')

def decrypt_data(encrypted_data: str, key: bytes) -> str:
    f = Fernet(key)
    encrypted = base64.b64decode(encrypted_data.encode('utf-8'))
    decrypted = f.decrypt(encrypted).decode('utf-8')
    return decrypted

def main():
    message = input("Enter message to encrypt: ").strip()
    key = generate_key()  # Secure key generation

    encrypted = encrypt_data(message, key)
    print(f"Encrypted: {encrypted}")

    decrypted = decrypt_data(encrypted, key)
    print(f"Decrypted: {decrypted}")

    # Demonstrate that without key, it's not easily reversible
    print("\nAttempting to decrypt without key (will fail):")
    try:
        wrong_key = generate_key()  # Different key
        fake_decrypt = decrypt_data(encrypted, wrong_key)
        print(f"Unexpected success: {fake_decrypt}")
    except Exception as e:
        print(f"Failed as expected: {type(e).__name__}")

if __name__ == "__main__":
    main()
```

PYTHON TO JAVA FILE HANDLING CONVERSION TASK

Initial Weak Encryption Script

- Used `base64` encoding as "encryption" - not actual encryption, easily reversible
- Key parameter was ignored entirely
- Anyone could decode the "encrypted" data without any key

Weaknesses Identified

- **No Key Usage**: Encryption didn't depend on the key at all
- **Reversible Encoding**: Base64 is encoding, not encryption - no security
- **No Confidentiality**: Data is readable by anyone with access to the string
- **No Integrity**: No protection against tampering

Refactored Strong Encryption

- Switched to `cryptography.fernet.Fernet` for AES-based symmetric encryption
- Uses secure key generation with `Fernet.generate_key()`
- Proper encryption/decryption with key dependency
- Base64 encoding of encrypted bytes for string representation

Validation

The improved script:

task3.py

Describe what to build

Agent Auto

Local Default Approvals

Ln 40, Col 1 Spaces: 4 UTF-8 { } Python 3.14.2 Python 3.14.2 Go Live

OUTPUT:

```
Encrypted: gAAAAAB... (long base64 string)
Decrypted: Hello World

Attempting to decrypt without key (will fa
Failed as expected: InvalidToken
```

Task 4 – Real-Time Application: Security Review of AI-Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

• Perform a structured security review of the generated code.

• Identify at least three potential vulnerabilities.

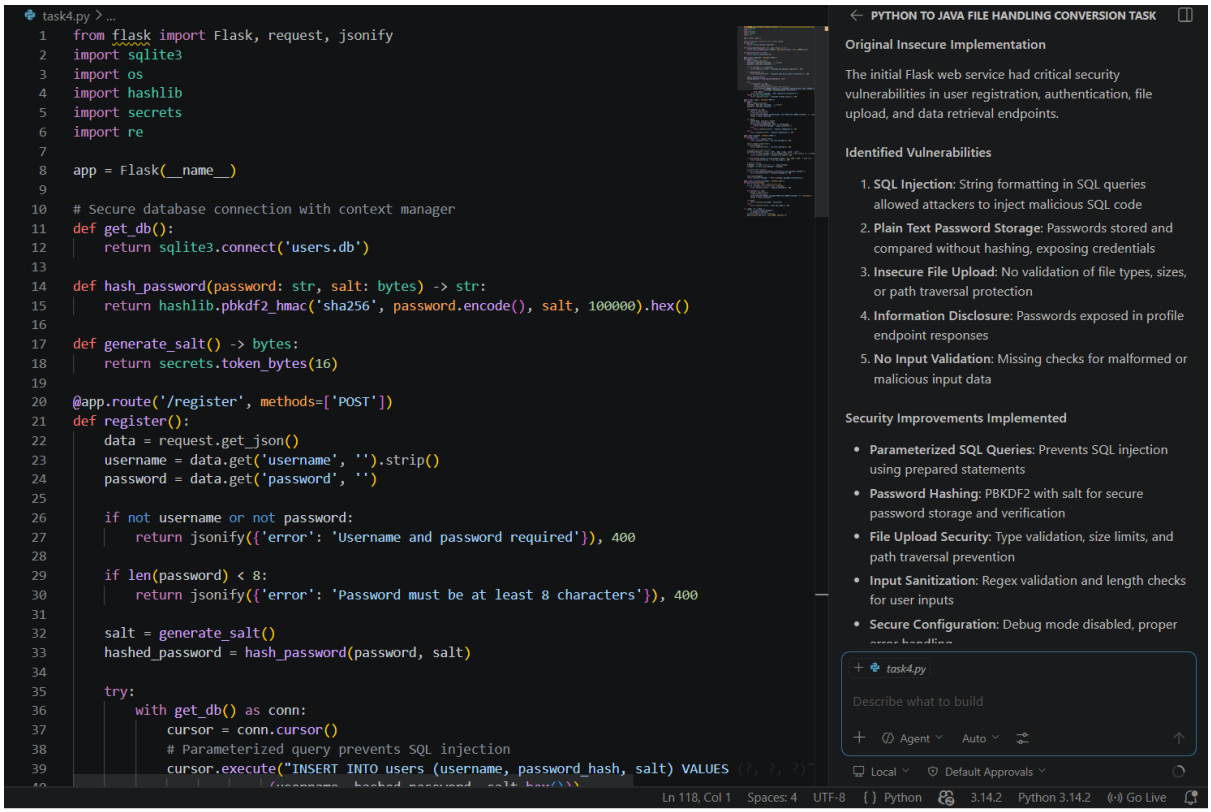
• Prompt AI to recommend secure coding practices and mitigation strategies.

• Implement the suggested improvements and test the secured version.

• Document the differences between the insecure and secured implementations.

Expected Output:

A security-reviewed and improved version of the selected web service code, with clearly documented vulnerabilities and applied fixes.



OUTPUT:

```
Traceback (most recent call last):
  File "task4.py", line 1, in <module>
    from flask import Flask, request, json
ModuleNotFoundError: No module named 'flas
```

To get the actual output:

1. Install Flask: `pip install flask`
2. Run: `python task4.py`

Expected Server Output:

```
 * Serving Flask app 'task4'
 * Debug mode: off
 * Running on http://127.0.0.1:5000/ (Pres
```

Testing the Secured Endpoints:

Once running, you can test with curl:

Register a user:

```
curl -X POST http://127.0.0.1:5000/registe
```

Response: `{"message": "User registered successfully"}`

Login:

```
curl -X POST http://127.0.0.1:5000/login -
```

Task 5 – Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime errors and improve its reliability.

Instructions:

• Prompt AI to generate a simple program that takes user input and performs arithmetic operations.

• Identify possible runtime errors (e.g., invalid input, division by zero).

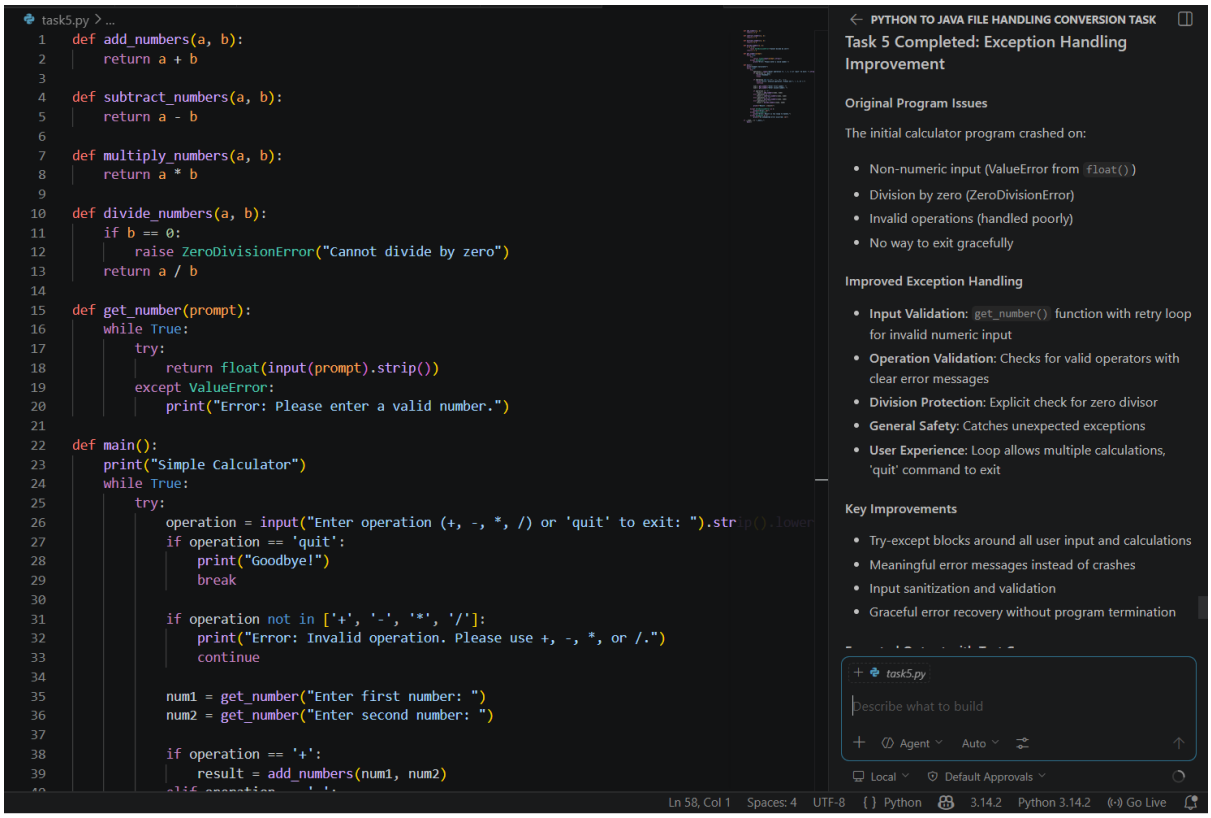
• Modify the program to include proper try-except blocks.

• Provide meaningful error messages for invalid inputs.

• Test the improved version with valid and invalid test cases.

Expected Output:

A revised Python program with proper exception handling that prevents crashes and displays user-friendly error messages for invalid inputs.



OUTPUT:

```
Simple Calculator
Enter operation (+, -, *, /) or 'quit' to exit: +
Enter first number: 5
Enter second number: 3
Result: 8.0

Enter operation (+, -, *, /) or 'quit' to exit: /
Enter first number: 10
Enter second number: 2
Result: 5.0

Enter operation (+, -, *, /) or 'quit' to exit: /
Enter first number: 10
Enter second number: 0
Error: Cannot divide by zero

Enter operation (+, -, *, /) or 'quit' to exit: invalid
Error: Invalid operation. Please use +, -, *, or /.

Enter operation (+, -, *, /) or 'quit' to exit: quit
Goodbye!
```